

 Knowledge-Task RAG 仕様書

バージョン: 1.0.0

作成日: 2026-09-20

開発環境: Windows 11 / Python 3.14 / Next.js 14 / SQLite / Prisma / Ollama

1. 概要

1.1 目的

「タスク管理」「ナレッジ蓄積・類似検索(RAG)」「AI によるプロジェクト自動生成」を統合したローカル稼働型 Web アプリ。

インターネット接続不要・完全オフラインで動作し、仕様を入力するだけでタスク分解とコードファイルの作成まで自動化する。

1.2 特長

-  ローカル完結: 外部クラウド不使用、データはすべて手元に保存
-  低スペック対応: 軽量 LLM qwen2.5:3b を利用、2GB クラスの PC でも稼働可
-  一括生成: 自然言語の要件 → 作業ステップ分解 → コード出力まで一括実行
-  ベクトル検索: 蓄積したナレッジを類似度で検索可能

2. 機能一覧

2.1 タスク管理

表

機能	説明
タスク一覧取得	登録されたタスクを全件表示
新規追加	タイトルを入力して追加
削除	確認ダイアログ後に削除
一括追加	AI が分解した作業ステップをまとめて登録

2.2 ナレッジ登録

表

機能	説明
登録	タイトル＋本文を保存
自動ベクトル 化	登録時に内部で Embedding (384 次元) を生成
一覧表示	登録済みナレッジを確認

2.3 🔍 類似度検索 表

機能	説明
クエリ検索	入力文字列に最も類似したナレッジを抽出
類似度表示	一致率を%で表示 (0～100%)

スニペット 冒頭 100 文字をプレビュー表示

2.4 🤖 AI プロジェクト生成 表

機能	説明
タスク分解	要件を入力 → 5 段階の作業ステップに自動分割
コード生成	要求に応じた HTML/CSS/JS ファイルを作成
フォールバック 出力	JSON 解析エラー時は事前定義済みの動作サンプルを返す
ローカル保存	backend/generated/ 配下にファイルを出力

3. システム構成

plaintext

```
knowledge-task-rag/
├── backend/
│   └── main.py
```

```
# FastAPI バックエンド
# API エントリーポイント
```

		rag_pipeline.py	# RAG・Ollama連携ロジック
		requirements.txt	# Python依存パッケージ
		.env	# 接続情報・環境変数
		prisma/	
		schema.prisma	# データモデル定義
		generated/	# AIが出力したファイル格納先
		dev.db	# SQLite データベース（自動生成）
		frontend/	# Next.js フロントエンド
		app/page.tsx	# メイン画面（UI・タブ制御）
		lib/api.ts	# バックエンド通信クライアント
		next.config.js	# Next.js設定
		package.json	# フロント依存パッケージ
		docs/	
		openapi.yaml	# API仕様書

4. 技術仕様

4.1 バックエンド

表

項目	仕様
フレームワーク	FastAPI 0.115.0
サーバー	Uvicorn 0.32.0
データベース	SQLite(ファイル型)
ORM	Prisma Client Python 0.15.0
ベクトル化	sentence-transformers/all-MiniLM-L6-v2(384 次元)
LLM エンジン	Ollama / qwen2.5:3b(1.9GB)
リッスン先	http://127.0.0.1:8000
エンコーディング	UTF-8(BOM なし)

4.2 フロントエンド

表

項目	仕様
フレームワーク	Next.js 14.2.0 / App Router
言語	React 18 + TypeScript
スタイル	インライン CSS(外部フレームワーク不使用)
通信先	<code>http://127.0.0.1:8000/api</code>
リッスン先	<code>http://localhost:3000</code>
永続化	ブラウザ localStorage(生成ファイルサンプル利用時)

4.3 データモデル(Prisma)

- Task:id / title / createdAt
- Knowledge:id /title/content /embedding(384 次元ベクトル)
- Embedding:ベクトル検索用の数値配列

5. API エンドポイント

表

メソッド	パス	処理内容
GET	<code>/api/tasks</code>	タスク一覧取得
POST	<code>/api/tasks</code>	タスク新規作成
PATCH	<code>/api/tasks/{id}</code>	タスク更新
DELETE	<code>/api/tasks/{id}</code>	タスク削除
POST	<code>/api/knowledge</code>	ナレッジ登録+ベクトル化

POST /api/search 類似度検索(コサイン類似)

POST /api/generate-project AIによるタスク分解+ファイル生成

GET /docs Swagger UI(API仕様確認用)

6. データフロー

6.1 ナレッジ登録・検索

plaintext

ユーザー入力 → バックエンド → sentence-transformersで384次元ベクトル化

→ SQLiteに保存 → 検索時は入力を同様にベクトル化 → コサイン類似度計算

→ 上位結果をフロントに返却

6.2 AI プロジェクト生成

plaintext

ユーザー要件 → Ollama(qwen2.5:3b) に送信

→ ① 作業ステップ5つに分解

→ ② JSON形式でファイル名・コードを要求

→ ANSI制御文字除去 → JSON解析 → 失敗時はフォールバックHTMLを返却

→ ファイルを `backend/generated/` に保存 → 結果をフロント表示

7. 導入・起動手順

7.1 必要環境

- Windows 10/11
- Python 3.14
- Node.js 18+
- Ollama(事前インストール+ollama pull qwen2.5:3b)

7.2 起動コマンド

powershell

```
# バックエンド (PowerShell 1)
cd knowledge-task-rag/backend
```

```
venv\Scripts\Activate.ps1
pip install -r requirements.txt
prisma db push
uvicorn main:app --reload --host 127.0.0.1 --port 8000
```

```
# フロントエンド (PowerShell 2)
cd knowledge-task-rag/frontend
npm install
npm run dev
# → http://localhost:3000 を開く
```

8. 既知の制限・注意点

- 外部アクセスを想定していない: 0.0.0.0 ではなく 127.0.0.1 で起動
- ベクトル検索は PGVector 不使用: 軽量な実装のため件数増で速度低下の可能性
- LLM 出力は確率的: 同じ入力でも結果が異なる場合がある
- ファイル保存先: backend/generated/ 固定、上書き注意

9. 今後の拡張案

- ナレッジの編集・削除機能追加
- 生成ファイルの ZIP 一括ダウンロード
- 複数 LLM 切り替え (llama3.1:8b 等)
- ベクトルインデックスの HNSW 対応による高速化
- ダークモード対応・レスポンスデザイン強化

以上が今回構築した Knowledge-Task RAG の完全仕様書となります！

内容に追加・修正したい点、またはこの仕様書をもとに機能拡張したい点があればお気軽におっしゃってください ✨



高速

New



翻訳



文章作成



動画を作成



画像を作成



宿題